

Web & Network Basics, Spring Boot Startup Internals & Redis Caching

The Complete Guide — From "How the Internet Works" to Production-Grade Caching

IP Addresses, Cookies & Sessions • Spring Boot Startup Sequence (21 Steps) • Request Lifecycle

What Is Caching, Before/After Redis, Real Implementation, Interview Q&A

A practical, simple-language, end-to-end reference for interview preparation and real-world implementation

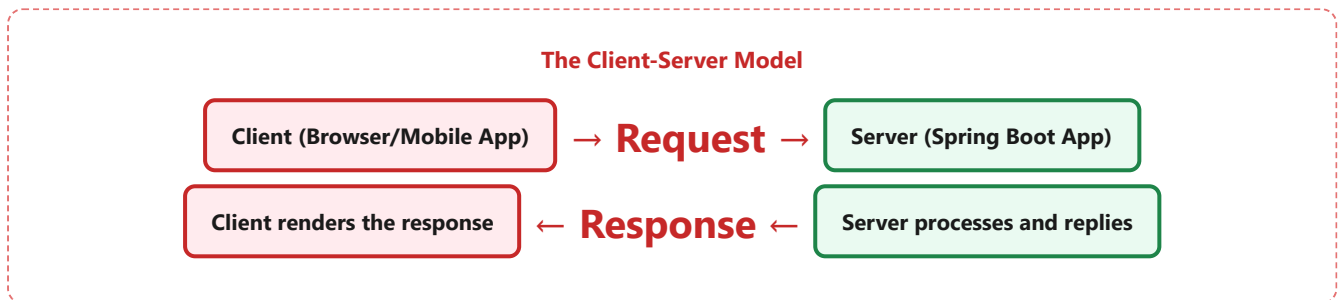
Table of Contents

1. Web Basics — How the Internet Actually Works
2. Network Basics — IP Addresses, DNS & Ports
3. HTTP and HTTPS — The Web's Communication Protocol
4. Basic Website Flow — From Typing a URL to Seeing a Page
5. Cookies — What, Why and How
6. Sessions — What, Why, How, and Session vs Cookie
7. How Login & Authentication Work Internally
8. Real-Time Network Concepts — Load Balancer, Reverse Proxy, CDN, Firewall
9. Web & Network Basics — Interview Questions & Real Scenarios
10. Spring Boot Application Startup — The Full Journey Overview
11. Step 1-2: JVM Starts & Main Method Executes
12. Step 3-4: SpringApplication.run() & ApplicationContext Created
13. Step 5-6: Environment Loaded & application.properties/YAML Loaded
14. Step 7-9: Component Scan, Bean Definitions & Dependency Injection
15. Step 10: Auto Configuration Starts
16. Step 11-12: DataSource Created & JPA/Hibernate Initialized
17. Step 13: Security Filters Loaded
18. Step 14-16: DispatcherServlet Created, Embedded Tomcat Starts & Port Bound
19. Step 17-19: Bean Initialization Completed, @PostConstruct & CommandLineRunner
20. Step 20-21: ApplicationReadyEvent Published & Application Ready
21. What Happens When a Request Hits — The Full Real-Time Request Lifecycle
22. Spring Boot Startup & Request Lifecycle — Interview Questions & Real Scenarios
23. What Is Caching and Why We Need It
24. Before Redis — What We Used Instead
25. Introduction to Redis
26. Redis Data Structures
27. Redis Architecture — How It Actually Works Internally
28. Implementing Redis in Spring Boot — Step by Step
29. Redis Use Cases Beyond Simple Caching
30. Cache Eviction Policies & Caching Strategies (Cache-Aside, Write-Through, Write-Behind)
31. Distributed Caching Challenges — Consistency & Invalidation
32. Redis High Availability — Replication, Sentinel & Cluster
33. Redis Security Best Practices
34. Redis Real-Time Issues & Scenarios
35. Redis Interview Questions & Answers
36. Final Summary & Cheat Sheet

1. Web Basics — How the Internet Actually Works

1.1 The Client-Server Model

Almost everything on the web follows the same basic pattern: a **Client** (your browser, mobile app) sends a **Request** to a **Server** (a remote computer running your application), and the server sends back a **Response**. This simple request/response cycle is the foundation of everything else in this guide.



1.2 What Actually Happens Between "Client" and "Server"

The client and server are rarely directly connected — the request travels across the physical Internet, through many intermediate devices (routers, switches, ISPs, potentially load balancers and firewalls) before reaching the actual server. This entire journey typically takes just milliseconds.

1.3 Frontend vs Backend (Quick Recap)

Layer	Runs Where	Real Example
Frontend	In the client's browser (HTML/CSS/JavaScript, React)	Renders the UI, handles user interaction
Backend	On the server (Spring Boot, Node.js)	Business logic, database access, security
Database	On the server (often a separate machine)	Persists data long-term

This guide builds up layer by layer — from the network fundamentals underneath every web request, through exactly how a Spring Boot application starts up and handles a request internally, to how Redis caching fits into that same real-time flow.

2. Network Basics — IP Addresses, DNS & Ports

2.1 What an IP Address Is

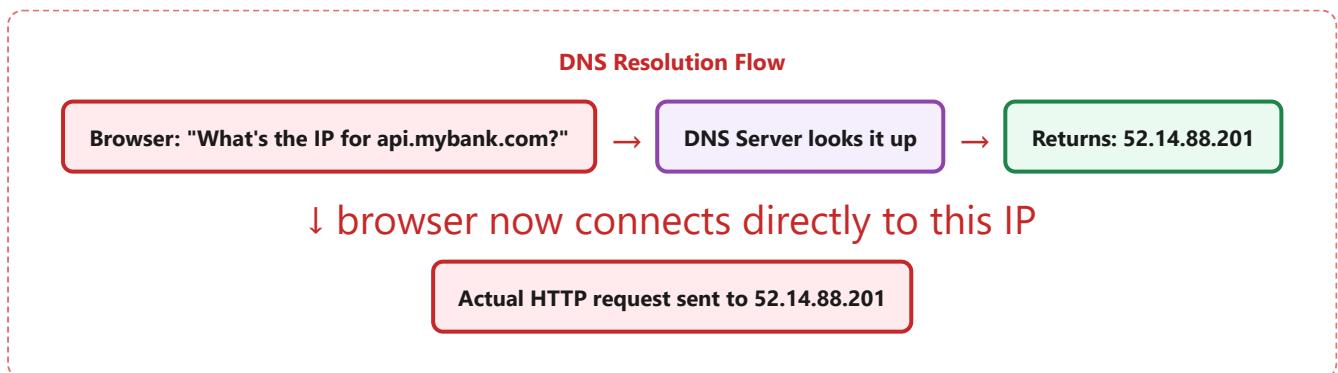
An IP (Internet Protocol) Address is a unique numerical identifier assigned to every device on a network — it's how one computer knows exactly WHERE to send data on the internet, similar to how a street address tells a postal service where to deliver a letter.

IPv4 example: 192.168.1.10 # 4 numbers (0-255), the older, still widely used format
IPv6 example: 2001:0db8:85a3::8a2e:0370:7334 # newer format, vastly more available addresses

IP Type	Meaning	Real Example
Public IP	Unique across the entire internet, reachable from anywhere	Your company's server's internet-facing address
Private IP	Only unique/reachable within a local network (behind a router/NAT)	Your laptop's address on your office/home Wi-Fi (192.168.x.x)
Static IP	Manually assigned, never changes	A production database server's fixed address
Dynamic IP	Automatically assigned, can change over time (via DHCP)	Most home internet connections

2.2 DNS — Translating Names Into IP Addresses

Humans use domain names (`www.mybank.com`) because they're memorable; computers actually need an IP address to connect. DNS (Domain Name System) is the "phone book of the internet" — it translates a domain name into the actual IP address of the server hosting it.



2.3 Ports — Identifying a Specific Application on a Server

An IP address identifies a MACHINE; a Port identifies a SPECIFIC APPLICATION/SERVICE running on that machine — one server can run many different applications simultaneously, each listening on its own port.

Port	Commonly Used By
------	------------------

80	HTTP (unencrypted web traffic)
443	HTTPS (encrypted web traffic) — the modern default for virtually all real websites
8080	Common default for Spring Boot's embedded Tomcat during development
5432	PostgreSQL database
3306	MySQL database
6379	Redis (Section 25+)

Real scenario: "When our React app calls `https://api.mybank.com/accounts/1001` , the browser first resolves `api.mybank.com` to an IP via DNS, then connects to that IP on port 443 (HTTPS), where our load balancer is listening and forwards the request internally to one of our Spring Boot pods running on port 8080."

3. HTTP and HTTPS — The Web's Communication Protocol

3.1 What HTTP Is

HTTP (HyperText Transfer Protocol) is the standard set of rules for how a client and server exchange requests and responses on the web — defining request methods (GET, POST, etc.), status codes, and headers (already covered in full depth in the API Development guide).

3.2 A Real HTTP Request/Response, Raw Format

```
--- Real Request ---
GET /accounts/1001 HTTP/1.1
Host: api.mybank.com
Authorization: Bearer eyJhbGciOi...
Accept: application/json

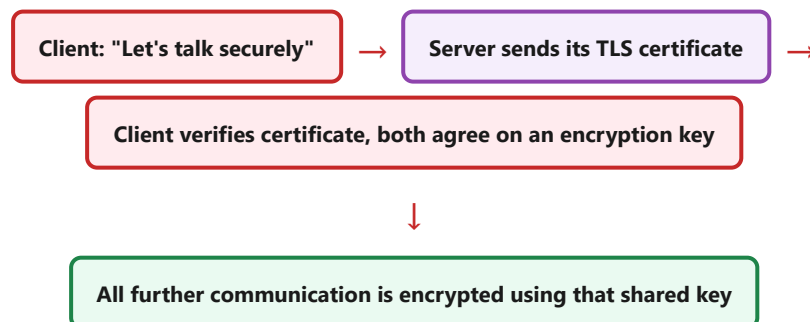
--- Real Response ---
HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: JSESSIONID=A3F8B2C1D9; HttpOnly; Secure

{"accountId": 1001, "balance": 4500.00}
```

3.3 HTTPS — HTTP Secured With Encryption

HTTPS is HTTP layered on top of TLS/SSL encryption — everything exchanged between client and server is encrypted in transit, so anyone intercepting the traffic (on public Wi-Fi, for example) sees only unreadable encrypted bytes, not your actual data (passwords, account balances, tokens).

The TLS Handshake (Simplified)

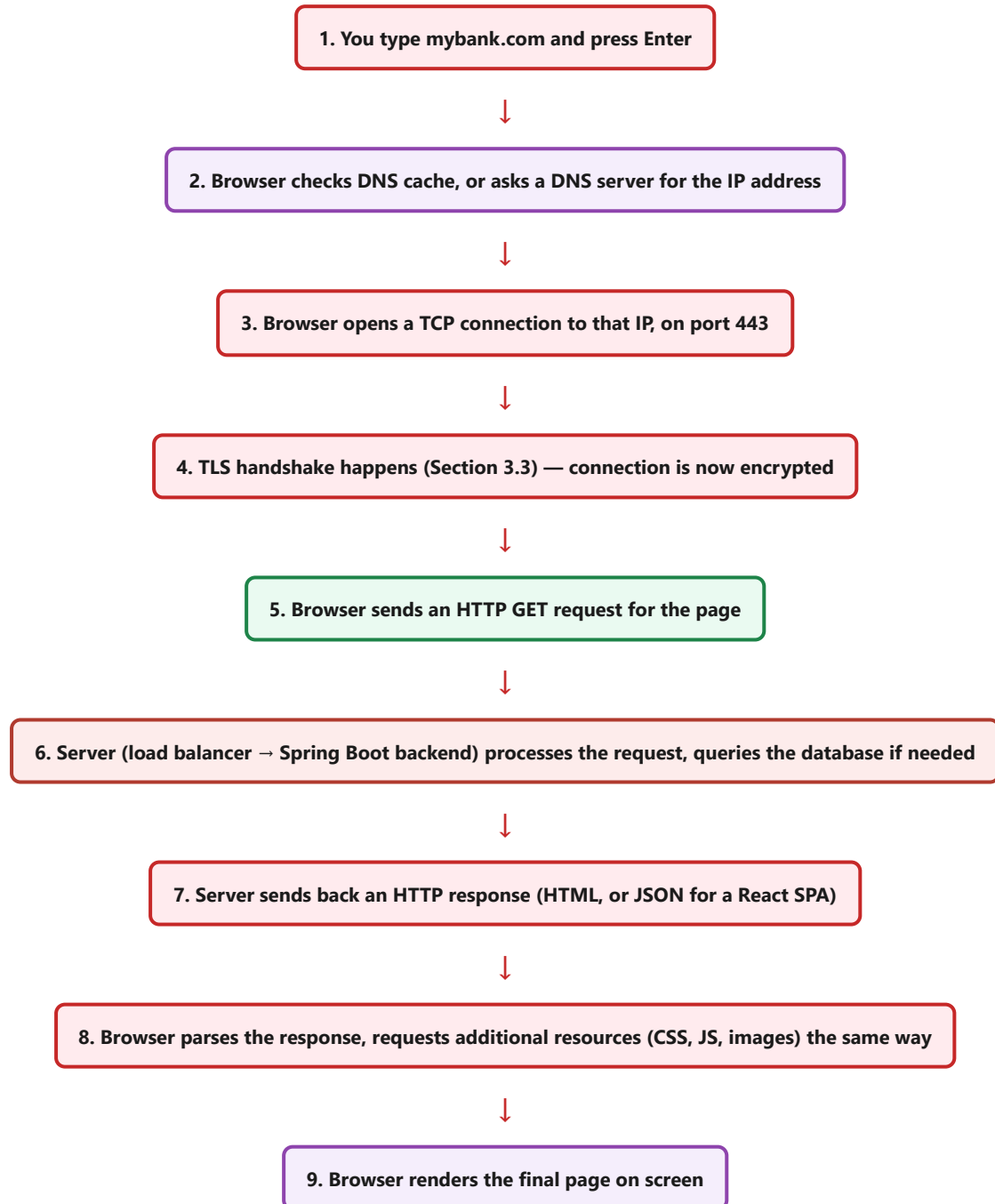


Real-world rule: Never transmit sensitive data (passwords, tokens, financial information) over plain HTTP — always HTTPS. Modern browsers actively warn users ("Not Secure") when a site doesn't use HTTPS, and real production systems enforce HTTPS-only with no HTTP fallback at all.

4. Basic Website Flow — From Typing a URL to Seeing a Page

4.1 The Complete Real Flow, Step by Step

What Really Happens When You Visit a Website



4.2 Real Modern Single Page Application (SPA) Flow

For a React app specifically: step 7 above initially returns a mostly-empty HTML shell plus JavaScript bundle. The browser then executes that JavaScript, which makes SEPARATE API calls (like `GET /api/v1/accounts/1001`) to fetch actual data as

JSON, and React renders the UI dynamically in the browser — this is the same request/response model, just happening multiple times after the initial page load, entirely orchestrated by JavaScript.

5. Cookies — What, Why and How

5.1 The Problem Cookies Solve — HTTP Is Stateless

HTTP is fundamentally **stateless** — by design, the server treats every single request as completely independent, with no memory of any previous request. Without something extra, a server has no way to know "this request and that earlier request came from the same logged-in user."

5.2 What a Cookie Is

A Cookie is a small piece of data the SERVER asks the BROWSER to store, which the browser then automatically sends back with every subsequent request to that same domain — giving the server a way to recognize the same client across multiple otherwise-independent requests.

```
-- Server sets a cookie in its response --
HTTP/1.1 200 OK
Set-Cookie: JSESSIONID=A3F8B2C1D9; HttpOnly; Secure; SameSite=Strict; Max-Age=1800

-- Browser automatically includes it on every future request to the same domain --
GET /accounts/1001 HTTP/1.1
Cookie: JSESSIONID=A3F8B2C1D9
```

5.3 Key Cookie Attributes

Attribute	Purpose
HttpOnly	JavaScript cannot access this cookie at all — protects against theft via XSS attacks
Secure	The cookie is only ever sent over HTTPS, never plain HTTP
SameSite	Controls whether the cookie is sent on cross-site requests — helps prevent CSRF attacks
Max-Age / Expires	How long the cookie lives before the browser automatically deletes it
Domain / Path	Restricts which domain/URL paths the cookie is sent to

5.4 What Cookies Are Actually Used For

- Session identification (Section 6) — the single most common real use.
- Remembering user preferences (theme, language) across visits.
- Tracking/analytics (with user consent, per privacy regulations like GDPR).
- Shopping cart contents for guest (non-logged-in) users on e-commerce sites.

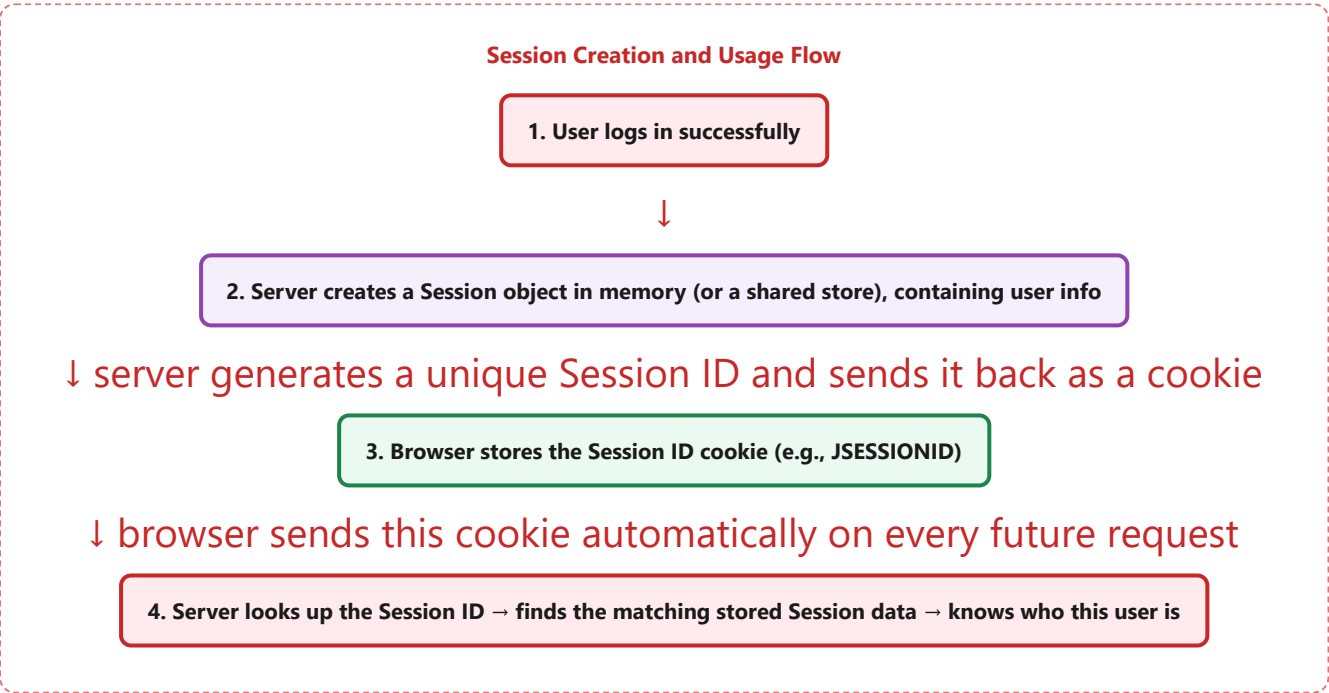
Interview tip: A cookie itself is just a small piece of data storage on the client, controlled and requested by the server — it's a general mechanism, and "session cookies" (Section 6) are simply the most common specific USE of that mechanism.

6. Sessions — What, Why, How, and Session vs Cookie

6.1 What a Session Is

A Session is server-side storage of information about a specific user's ongoing interaction with the application (e.g., "this user is logged in as customer #55") — the SERVER stores this data, and the CLIENT only holds a small session ID (usually via a cookie) that references it.

6.2 How Sessions and Cookies Work Together



6.3 Session vs Cookie — The Key Distinction

	Cookie	Session
Where stored	Client (browser)	Server (memory, database, or Redis — Section 25+)
What it holds	Small data, including (often) just the session ID itself	The actual user data — full profile, permissions, cart contents, etc.
Security exposure	Visible/accessible on the client (though HttpOnly limits JS access)	Never directly exposed to the client at all — only the reference ID is
Size limits	Small (a few KB per cookie, browser-enforced)	Can hold much more data — limited only by server memory/storage

6.4 Sessions in a Multi-Instance/Microservices World — A Real Problem

If a Spring Boot application stores sessions in local server memory (the default), and you run 4 replicas behind a load balancer, a user's session only exists on the ONE instance that created it — if their next request is routed to a DIFFERENT instance, that instance has never heard of their session, and they appear logged out. This is exactly why real production systems externalize session storage.

```
<dependency>
  <groupId>org.springframework.session</groupId>
  <artifactId>spring-session-data-redis</artifactId> // stores sessions in Redis – shared across ALL
instances
</dependency>
```

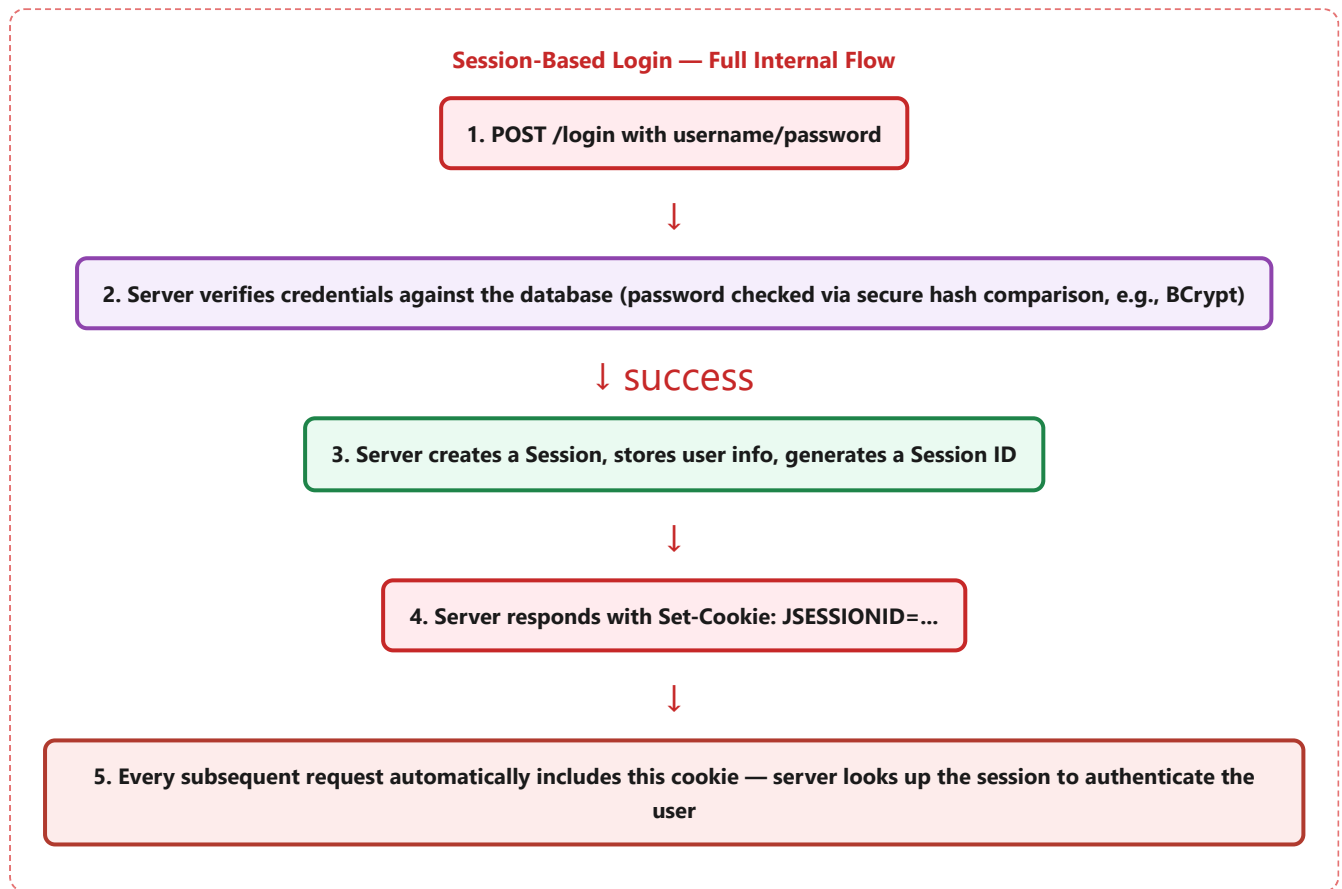
Real scenario: "Before we externalized sessions to Redis (Spring Session), users occasionally got randomly logged out mid-use — traced to our load balancer routing their next request to a different pod that had no record of their in-memory session. Moving sessions to Redis (shared across all pods) fixed this completely, since ANY pod can now look up ANY user's session."

6.5 The Modern Alternative — Token-Based (JWT) Authentication

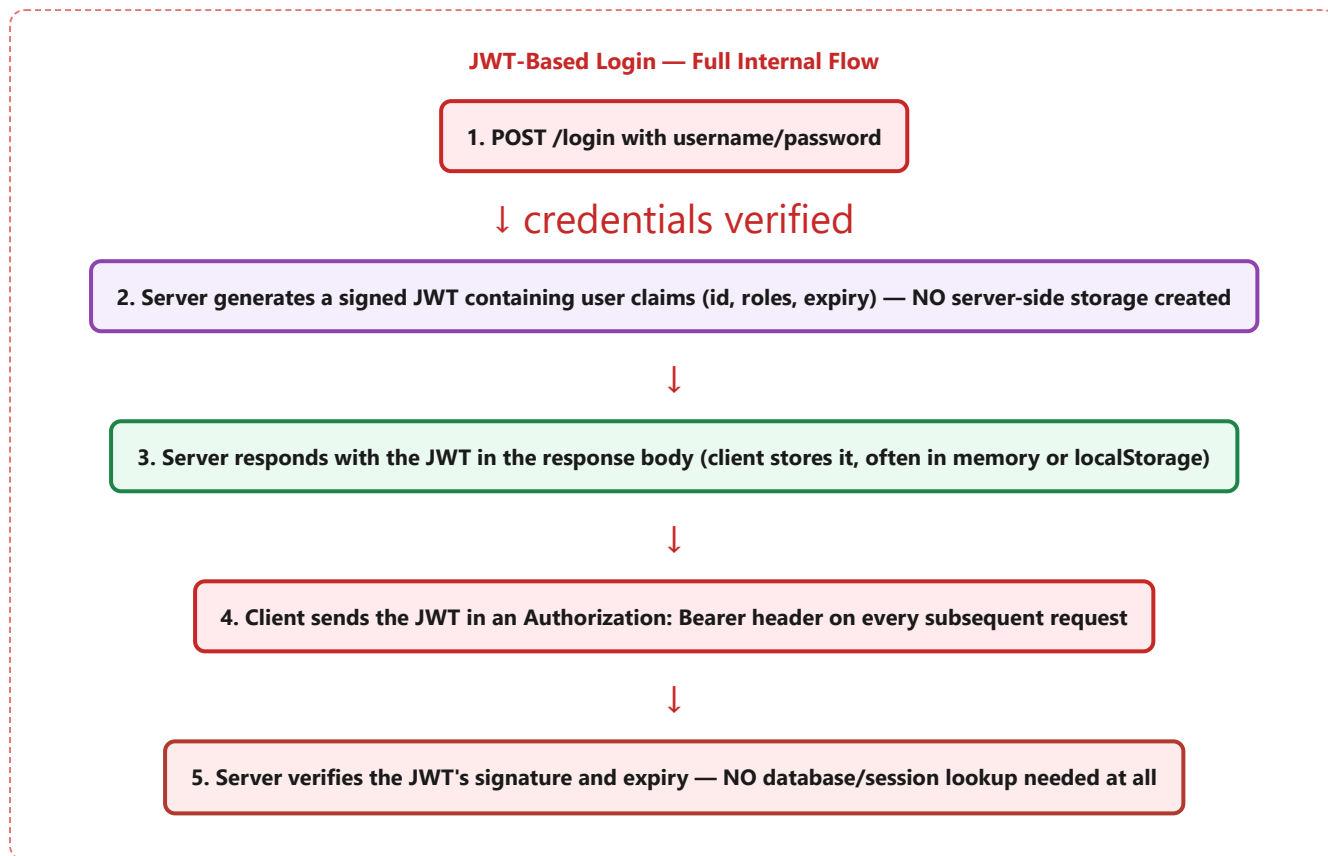
Many modern REST APIs avoid server-side sessions entirely, using stateless JWTs instead (full detail in the API Development guide, Section 18) — the token itself carries the user's identity/claims, verified by signature, with NOTHING stored server-side at all. This sidesteps the multi-instance session problem entirely, at the cost of harder immediate token revocation.

7. How Login & Authentication Work Internally

7.1 Traditional Session-Based Login Flow



7.2 Modern Token-Based (JWT) Login Flow



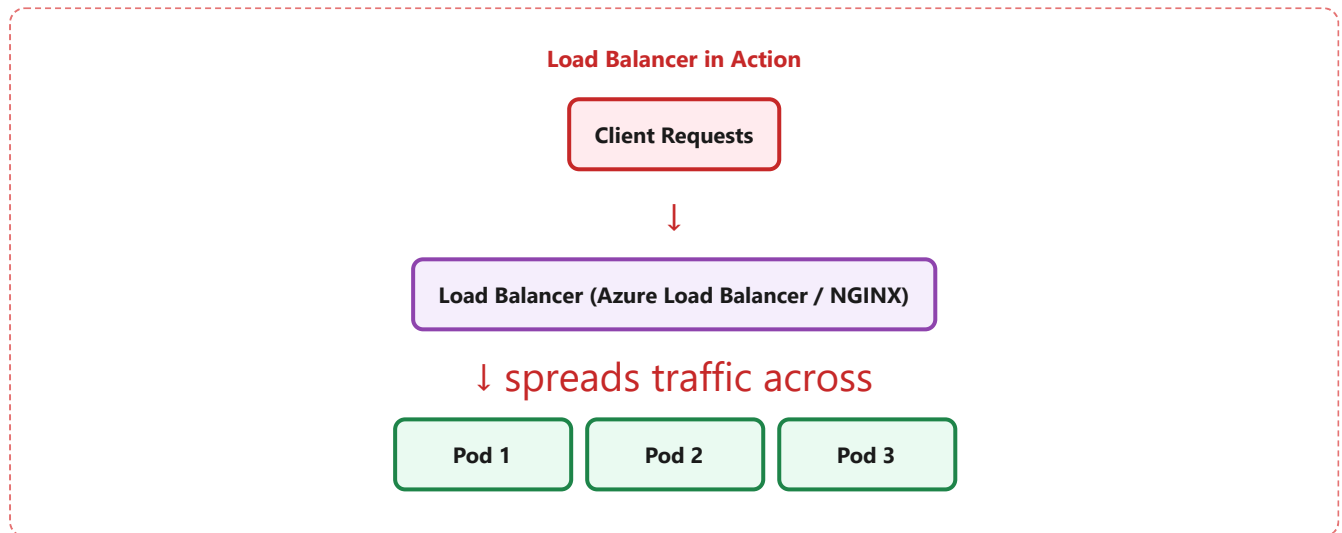
7.3 Session-Based vs Token-Based — Full Comparison

Aspect	Session-Based (Cookies)	Token-Based (JWT)
Server state	Stateful — server stores session data	Stateless — server stores nothing, verifies signature only
Scaling across instances	Requires shared session store (Redis) or sticky sessions	Naturally scales — any instance can verify any token independently
Revocation	Easy — just delete the session server-side	Harder — a signed token remains valid until it expires, unless you maintain a blocklist
Typical use	Traditional server-rendered web apps	Modern REST APIs, mobile apps, microservices

8. Real-Time Network Concepts — Load Balancer, Reverse Proxy, CDN, Firewall

8.1 Load Balancer

Distributes incoming requests across multiple backend server instances, so no single server gets overwhelmed and traffic keeps flowing even if one instance fails.



8.2 Reverse Proxy

Sits in front of backend servers, forwarding client requests to the appropriate backend and returning the response — often combined with load balancing, TLS termination, caching, and request routing (NGINX, our own API Gateway from the Microservices guide).

8.3 CDN (Content Delivery Network)

A globally distributed network of servers caching static content (images, CSS, JS, videos) physically close to the end user — dramatically reducing load times by serving content from a nearby server instead of one central origin server halfway around the world.

8.4 Firewall

A security barrier that filters network traffic based on rules — blocking unauthorized access attempts, restricting which ports/IPs can reach your servers, and forming a critical layer of real production network security.

Real scenario: "A request to our banking app's static assets (logo, CSS) is served instantly from Azure CDN edge locations worldwide, while the actual API call for account data goes through Azure Front Door (reverse proxy + WAF/firewall) → Load

Balancer → one of our AKS pods — different parts of the same page load use entirely different pieces of this real network infrastructure."

9. Web & Network Basics — Interview Questions & Real Scenarios

Q1. What is the difference between an IP address and a domain name?

An IP address is the actual numerical address identifying a device on a network. A domain name (like mybank.com) is a human-readable alias that DNS translates into the corresponding IP address — computers ultimately still connect using the IP.

Q2. What is the difference between HTTP and HTTPS?

HTTPS is HTTP secured with TLS/SSL encryption — data in transit is encrypted, protecting it from interception. Modern applications should always use HTTPS, especially for anything involving authentication or sensitive data.

Q3. Why is HTTP described as "stateless," and what problem does that create?

Each HTTP request is handled independently, with no inherent memory of previous requests — this means the server has no built-in way to recognize that two requests came from the same logged-in user, which is exactly the problem cookies and sessions solve.

Q4. Explain the difference between a cookie and a session.

A cookie is a small piece of data stored on the CLIENT, automatically sent with requests. A session is data stored on the SERVER representing a user's state; the client typically only holds a session ID (often via a cookie) referencing that server-side data.

Q5. What real problem occurs with in-memory sessions in a multi-instance deployment, and how do you fix it?

If sessions are stored in each server instance's local memory, a user's session only exists on the instance that created it — routing their next request to a different instance makes them appear logged out. Fixed by externalizing session storage to a shared store like Redis (Spring Session), so any instance can access any user's session.

Q6. Why are modern REST APIs moving toward JWT/token-based auth instead of server-side sessions?

JWTs are stateless — the server verifies a signature without needing any shared session storage, letting the architecture scale naturally across any number of instances without the session-sharing problem. The trade-off is harder immediate token revocation compared to simply deleting a server-side session.

Q7. What are HttpOnly and Secure cookie flags, and why do they matter?

HttpOnly prevents JavaScript from accessing the cookie at all, protecting against theft via XSS attacks. Secure ensures the cookie is only ever transmitted over HTTPS, never plain HTTP — both are essential security defaults for any cookie carrying sensitive information like a session ID.

Q8. What role does a Load Balancer play in a real production system?

It distributes incoming client requests across multiple backend instances, preventing any single instance from being overwhelmed, and enables high availability by routing around any instance that becomes unhealthy or is taken down for a rolling deployment.

Q9. What's the difference between a CDN and a regular web server?

A CDN is a globally distributed network caching static content close to end users geographically, reducing latency for assets like images/CSS/JS. A regular web/application server typically runs from one (or a few) specific locations and handles dynamic, personalized content.

Q10. Walk through what happens, step by step, when a user visits a website.

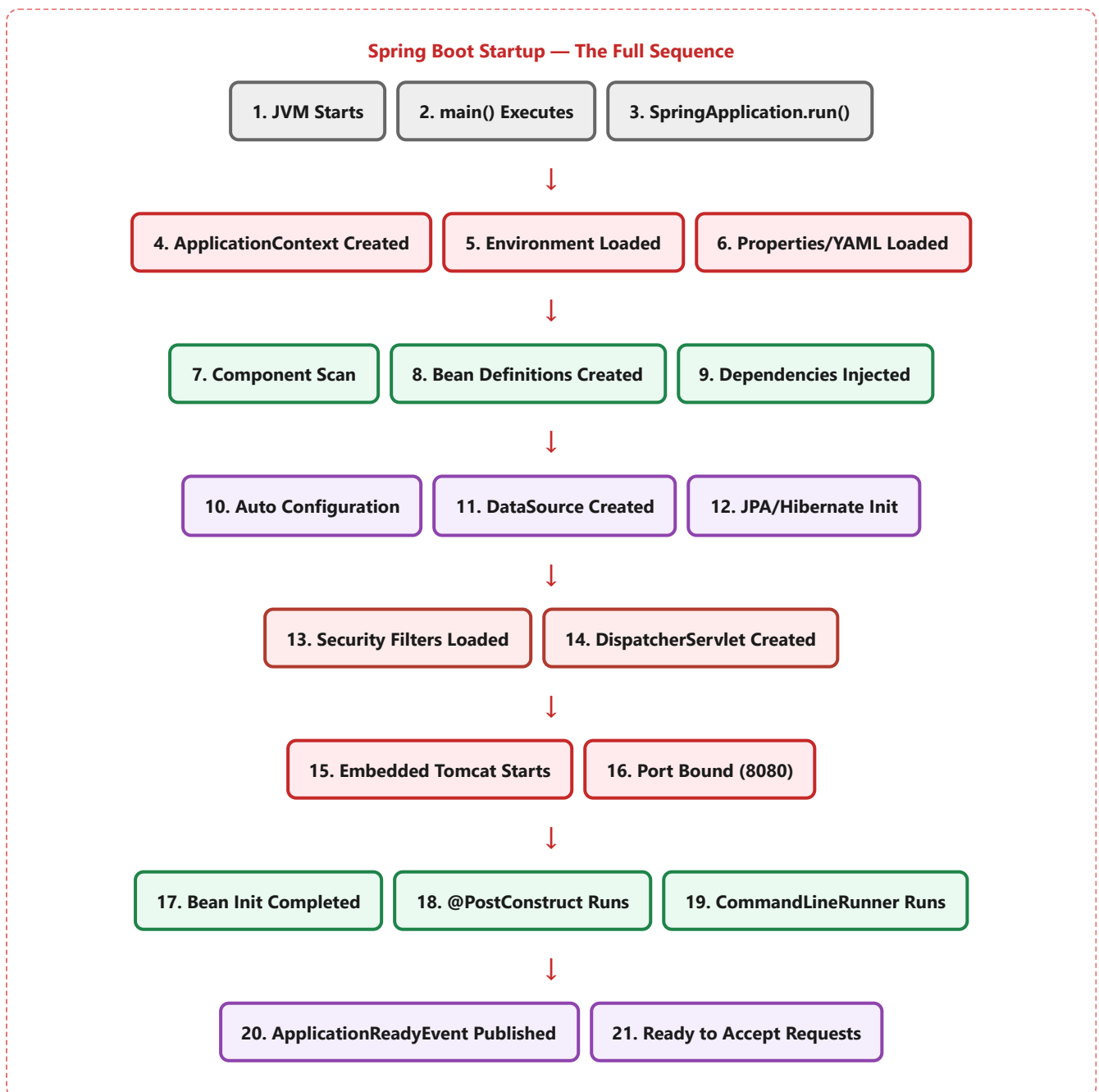
DNS resolves the domain to an IP → browser opens a TCP connection to that IP on the appropriate port → a TLS handshake establishes encryption (for HTTPS) → the browser sends an HTTP request → the server (often via a load balancer/reverse proxy) processes it, possibly querying a database → the server returns an HTTP response → the browser parses and renders it, fetching any additional referenced resources the same way.

10. Spring Boot Application Startup — The Full Journey Overview

10.1 Why Understanding Startup Internals Matters

Understanding exactly what happens between running `java -jar app.jar` and the application actually being ready to serve traffic helps you debug startup failures, understand bean wiring issues, optimize slow startup times, and answer one of the most common real interview questions: "explain what happens when a Spring Boot application starts."

10.2 The Complete 21-Step Journey, Visualized



The following sections walk through each of these 21 steps in real depth — the exact order matters, and understanding WHY each step happens where it does is what separates a memorized list from real understanding.

11. Step 1-2: JVM Starts & Main Method Executes

11.1 Step 1 — JVM Starts

Running `java -jar accounts-service.jar` launches the Java Virtual Machine — it loads the JVM runtime, allocates initial heap/stack memory, and locates the JAR's manifest to find the application's entry point (the class containing `public static void main`).

11.2 Step 2 — Main Method Executes

```
@SpringBootApplication
public class AccountsServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AccountsServiceApplication.class, args);    // this ONE line kicks off everything
        // else in this guide
    }
}
```

The `main` method is genuinely just plain Java — no Spring magic has happened yet at this exact point. The moment `SpringApplication.run()` is called is where the entire Spring Boot bootstrap process actually begins (Section 12).

12. Step 3-4: SpringApplication.run() & ApplicationContext Created

12.1 Step 3 — What SpringApplication.run() Actually Does

This single static method call orchestrates the ENTIRE startup sequence described in this guide — it's a facade hiding a large, well-defined sequence of internal steps (creating the context, loading configuration, scanning components, starting the embedded server, and more).

```
// Simplified view of what SpringApplication.run() does internally
public ConfigurableApplicationContext run(String... args) {
    // 1. Prepare the Environment (Section 13)
    // 2. Create the appropriate ApplicationContext type (web vs non-web)
    // 3. Refresh the context – triggers component scanning, bean creation, auto-configuration
    // 4. Run CommandLineRunners/ApplicationRunners
    // 5. Publish ApplicationReadyEvent
    // 6. Return the fully started context
}
```

12.2 Step 4 — ApplicationContext Created

The ApplicationContext is Spring's IoC container (full detail in the Spring & Spring Boot guide, Section 12) — for a typical web application, Spring Boot creates an `AnnotationConfigServletWebServerApplicationContext`, specifically aware of both annotation-based configuration AND the need to embed a web server.

Context Type Selection

spring-boot-starter-web on classpath?

→ YES →

ServletWebServerApplicationContext (embeds Tomcat)

NO → Plain AnnotationConfigApplicationContext (no embedded server, e.g., a batch job)

13. Step 5-6: Environment Loaded & application.properties/YAML Loaded

13.1 Step 5 — Environment Loaded

Spring builds an `Environment` object representing ALL the current configuration sources available — command-line arguments, JVM system properties, OS environment variables, and configuration files — establishing the precedence order covered in the Spring & Spring Boot guide (Section 35.2) before any actual file is even read.

13.2 Step 6 — application.properties/YAML Loaded

Spring Boot locates and parses `application.yml` / `application.properties` (and any active profile-specific variants like `application-prod.yml`) from the classpath, merging them into the Environment according to the documented precedence rules.

Configuration Loading Order

`application.yml (base defaults)`

→ **overridden by** →

`application-{active-profile}.yml`

→ **overridden by** →

`Environment variables / command-line args`

Real practical impact: This is exactly why `java -jar app.jar --server.port=9090` or `SPRING_PROFILES_ACTIVE=prod` can override anything defined in your YAML files — the Environment is assembled from multiple layered sources, with later/more specific sources winning, all resolved at this early stage before a single bean is even created.

14. Step 7-9: Component Scan, Bean Definitions & Dependency Injection

14.1 Step 7 — Component Scan Starts

`@SpringBootApplication` includes `@ComponentScan`, which tells Spring to scan the main application class's package AND all sub-packages, looking for classes annotated with `@Component`, `@Service`, `@Repository`, `@Controller`, `@Configuration`, and similar stereotypes.

14.2 Step 8 — Bean Definitions Created

For every class discovered during scanning (plus every `@Bean` method inside `@Configuration` classes), Spring creates a `BeanDefinition` — essentially a "recipe" describing how to construct that bean, its dependencies, and its scope — WITHOUT actually instantiating the object yet. This is a metadata-gathering phase.

Two-Phase Bean Creation

Phase 1: Gather ALL `BeanDefinitions` first (recipes, not real objects yet)



Phase 2: Instantiate actual bean objects, resolving dependencies between them

This two-phase approach is precisely why Spring can detect certain configuration problems (like circular dependencies) and resolve dependency ORDER correctly — it knows the full picture of what beans exist and what they need BEFORE creating any of them.

14.3 Step 9 — Dependencies Injected

Spring instantiates each bean (respecting dependency order — a bean needing another bean is created after its dependency), and injects dependencies via constructor injection (the recommended approach, per the Spring & Spring Boot guide Section 13) — resolving the entire object graph of your application.

15. Step 10: Auto Configuration Starts

15.1 What Happens at This Stage

Spring Boot's auto-configuration mechanism (full detail in the Spring & Spring Boot guide, Section 33) kicks in — scanning the classpath and conditionally registering additional beans (`DataSource`, `EntityManagerFactory`, security defaults, and more) that a typical application with these specific dependencies would need, WITHOUT you writing any explicit configuration.

```
// Real example: seeing spring-boot-starter-data-jpa + a DB driver on the classpath triggers:  
@Configuration  
@ConditionalOnClass(DataSource.class)  
@ConditionalOnMissingBean(DataSource.class)  
public class DataSourceAutoConfiguration { ... } // this is exactly what fires in Step 11
```

This step is why Spring Boot feels like "it just works" — dozens of these conditional auto-configuration classes are evaluated (and mostly skipped, if their conditions aren't met) at this exact point in startup, silently doing the configuration work you'd have written manually in plain Spring MVC (Spring & Spring Boot guide, Section 10).

16. Step 11-12: DataSource Created & JPA/Hibernate Initialized

16.1 Step 11 — DataSource Created

Using the connection details from `application.yml` (Step 6), Spring Boot's auto-configuration creates the actual `DataSource` bean — by default, a HikariCP connection pool, pre-establishing a configurable number of ready-to-use database connections.

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/accounts_db
    hikari:
      maximum-pool-size: 20    # HikariCP creates connections up to this pool size, ready before any request
                               arrives
```

16.2 Step 12 — JPA/Hibernate Initialized

With the `DataSource` now available, Spring Boot's JPA auto-configuration creates the `EntityManagerFactory` — Hibernate scans for `@Entity` -annotated classes, builds its internal metadata model of your entities/relationships, and (if `ddl-auto: validate` is set, per the JDBC/Hibernate/JPA guide Section 7.2) validates that your entity mappings actually match the real database schema.

Real startup failure scenario: If your `@Entity` class expects a column that doesn't exist in the actual database table, this EXACT step is where the application fails to start — with a clear `SchemaManagementException` — much better than discovering the mismatch later via a confusing runtime error on the first request that touches that entity.

17. Step 13: Security Filters Loaded

17.1 What Happens Here

If Spring Security is on the classpath, its auto-configuration builds the `SecurityFilterChain` — an ordered chain of servlet filters that will later intercept EVERY incoming request, handling authentication (JWT/OAuth2 validation, per the API Development guide Section 18) and authorization (role/permission checks) before the request ever reaches a controller.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health").permitAll()
            .anyRequest().authenticated()
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults())));
        return http.build(); // this chain is fully assembled and ready at THIS exact startup step
    }
}
```

This ordering matters — security filters are wired up and ready BEFORE the DispatcherServlet/embedded Tomcat even start accepting connections (Steps 14-16), guaranteeing no request can ever slip through before security is fully in place.

18. Step 14-16: DispatcherServlet Created, Embedded Tomcat Starts & Port Bound

18.1 Step 14 — DispatcherServlet Created

Spring Boot auto-configures and registers the `DispatcherServlet` (the Front Controller, full detail in the Spring & Spring Boot guide Section 22) — at this point it exists as a configured bean, but the actual embedded server that will route real network traffic to it hasn't started yet.

18.2 Step 15 — Embedded Tomcat Starts

Spring Boot programmatically starts the embedded Tomcat server (bundled inside your JAR, per the Spring & Spring Boot guide Section 38) — Tomcat initializes its own internal thread pools and connector infrastructure, and registers the `DispatcherServlet` as the handler for incoming requests.

18.3 Step 16 — Port Bound (8080)

Tomcat binds to the configured port (default 8080, or whatever `server.port` specifies) — this is the exact moment the operating system starts allowing TCP connections to reach your application on that port. Before this step, any connection attempt would simply be refused.

Steps 14-16 Together

DispatcherServlet registered (as a servlet definition)



Embedded Tomcat starts



Port 8080 bound — OS now routes incoming connections here

Important nuance: The port being bound does NOT yet mean the application is fully ready to correctly handle business requests — remaining beans still need to finish initializing (Step 17), and `@PostConstruct` / `CommandLineRunner` logic still needs to run (Steps 18-19). A request arriving in this narrow window could be accepted by Tomcat but hit a not-yet-fully-initialized application.

19. Step 17-19: Bean Initialization Completed, `@PostConstruct` & `CommandLineRunner`

19.1 Step 17 — Bean Initialization Completed

Any remaining beans not yet fully initialized (including ones that depend on the now-available `DataSource/EntityManagerFactory/Tomcat` infrastructure) complete their construction and dependency injection at this stage.

19.2 Step 18 — `@PostConstruct` Executes

For every bean, once its dependencies are fully injected, Spring calls any `@PostConstruct`-annotated method (full lifecycle detail in the Spring & Spring Boot guide, Section 14) — commonly used for cache warm-up, validating critical configuration, or initializing a connection to an external system.

```
@Component
public class InterestRateCacheWarmer {
    @PostConstruct
    public void warmUpCache() {
        log.info("Pre-loading interest rate table into cache...");
        // runs automatically, exactly here in the startup sequence, before the app is marked "ready"
    }
}
```

19.3 Step 19 — `CommandLineRunner` Executes

After the ENTIRE `ApplicationContext` has finished refreshing (all beans created, injected, and post-constructed), Spring Boot invokes any `CommandLineRunner` / `ApplicationRunner` beans (Spring & Spring Boot guide, Section 39) — the very last hook before the application is declared ready.

```
@Component
public class StartupDataVerifier implements CommandLineRunner {
    @Override
    public void run(String... args) {
        log.info("Verifying critical reference data is present...");
        // this runs AFTER every single bean in the whole application is fully ready
    }
}
```

20. Step 20-21: ApplicationReadyEvent Published & Application Ready

20.1 Step 20 — ApplicationReadyEvent Published

Once everything above has completed successfully, Spring publishes an `ApplicationReadyEvent` — any bean implementing `ApplicationListener<ApplicationReadyEvent>` (or using `@EventListener`) reacts to this signal, commonly used to log "Application started successfully" or trigger final health-check registrations.

```
@Component
public class StartupCompletionListener {
    @EventListener(ApplicationReadyEvent.class)
    public void onReady() {
        log.info("✅ Accounts Service is fully started and ready to accept traffic");
    }
}
```

20.2 Step 21 — Application Ready to Accept Requests

The application is now genuinely, fully ready — Tomcat is accepting connections, every bean is initialized, the security chain is active, the database connection pool is warm, and any startup verification logic has run successfully. This is precisely the state Kubernetes' readiness probe (`/actuator/health/readiness`, Spring & Spring Boot guide Section 20.4) is designed to detect before routing real user traffic to this pod.

Interview one-liner: "Startup isn't one single moment — it's a carefully ordered 21-step sequence where each step depends on the previous ones being complete: the JVM boots, the Spring container is built and populated with beans (with configuration loaded first), auto-configuration wires up infrastructure like the DataSource and security, the embedded web server starts and binds its port, and only after every bean's lifecycle hooks have run does Spring Boot declare the application truly ready."

21. What Happens When a Request Hits — The Full Real-Time

Request Lifecycle

21.1 The Complete Real-Time Request Flow

A Real Request's Full Internal Journey

1. Client sends HTTP request to the load-balanced public URL



2. Load Balancer / Ingress routes it to a healthy pod's Tomcat (port 8080)



3. Tomcat's connector accepts the TCP connection, hands off to a worker thread



4. Servlet Filter Chain runs FIRST — CORS filter, correlation ID filter, Spring Security's filter chain

↓ authentication/authorization checked HERE, before any business code runs

5. DispatcherServlet (Front Controller) receives the request



6. HandlerMapping determines which @Controller method matches the URL/HTTP method



7. HandlerInterceptors' preHandle() runs (if any registered)



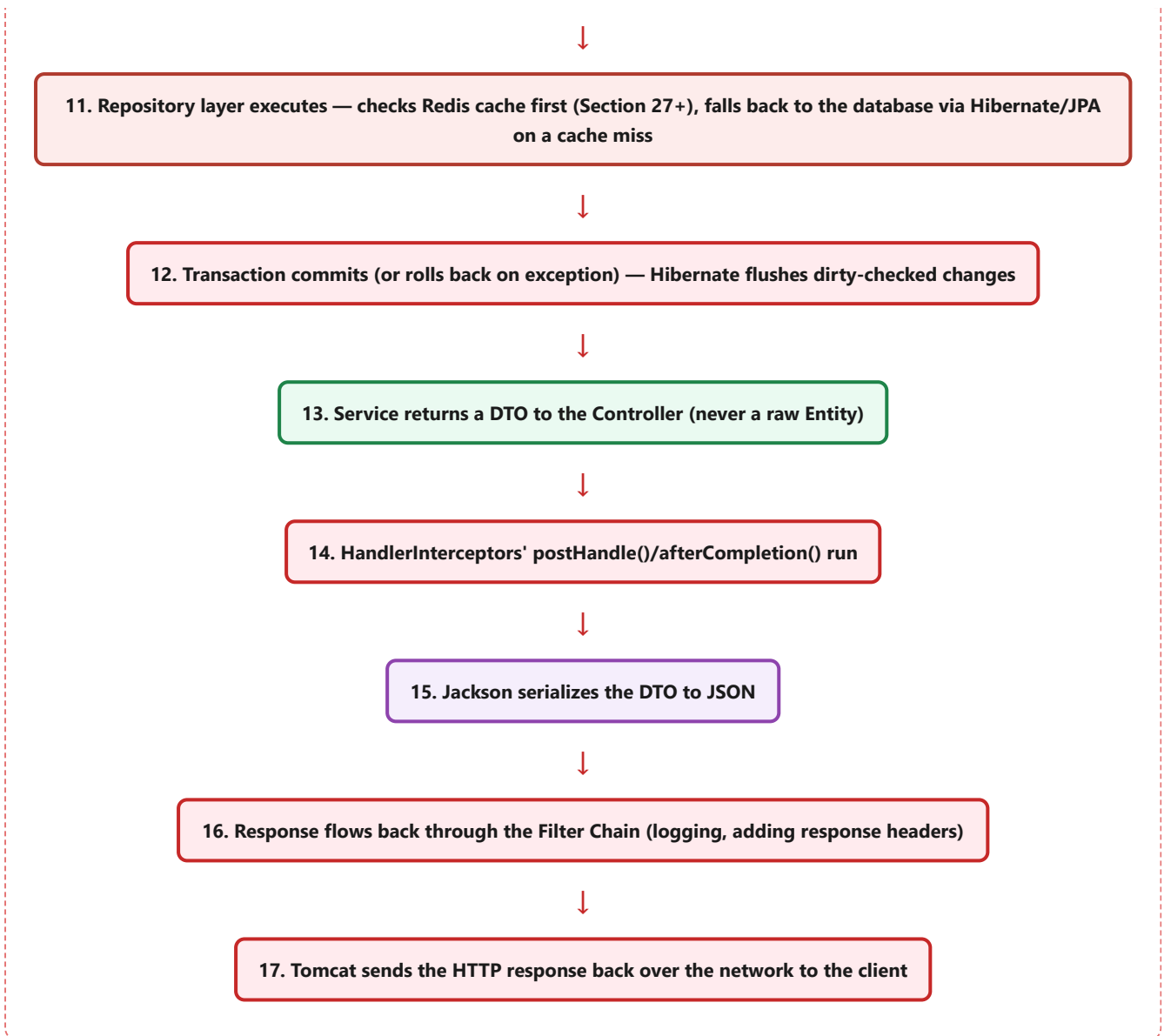
8. Request body deserialized to a DTO; @Valid triggers Bean Validation

↓ validation fails → GlobalExceptionHandler responds immediately, skipping remaining steps

9. Controller method executes, delegating to the Service layer



10. @Transactional boundary opens (if present) — Service layer runs business logic



21.2 Where Observability Hooks Into This Flow

At nearly every step above, cross-cutting observability is happening simultaneously (per the Observability & Monitoring guide): the correlation ID filter tags every log line, Micrometer records request latency/count metrics, and a distributed tracing agent (OpenTelemetry) creates a span for the whole request plus child spans for the DB query and any downstream calls — all without the business logic code needing to know any of this is happening.

21.3 Real Scenario Tying Startup and Request Handling Together

Real scenario: "During a rolling deployment, Kubernetes only sends traffic to a new pod once its readiness probe passes — which only happens after Step 21 of startup (Section 20.2) is genuinely complete. This guarantees no user request ever hits a pod where, say, the DataSource connection pool hasn't finished warming up yet, preventing a whole class of 'works eventually but fails on the first few requests after deploy' bugs."

22. Spring Boot Startup & Request Lifecycle — Interview Questions & Real Scenarios

Q1. Walk through what happens when a Spring Boot application starts, end to end.

JVM starts and executes `main()` → `SpringApplication.run()` creates the `ApplicationContext` → Environment and `application.yml` are loaded → component scanning creates bean definitions → dependencies are injected → auto-configuration wires infrastructure (`DataSource`, JPA, Security) → `DispatcherServlet` is created and embedded Tomcat starts, binding the port → remaining beans finish initializing, `@PostConstruct` and `CommandLineRunner` execute → `ApplicationReadyEvent` is published and the app is ready to serve traffic.

Q2. Why is the Environment/configuration loaded so early, before beans are created?

Bean creation (especially auto-configuration) depends on knowing configuration values (like `datasource` URLs, active profiles) upfront to decide what to configure and how — configuration must be fully resolved before Spring can correctly build the bean graph that depends on it.

Q3. What's the difference between when BeanDefinitions are created versus when beans are actually instantiated?

`BeanDefinitions` are metadata "recipes" gathered during component scanning describing how to construct each bean and what it depends on — created first, in bulk, without instantiating anything. Actual bean instantiation happens afterward, in dependency order, using those recipes.

Q4. Why does JPA/Hibernate initialization happen after DataSource creation, and what can go wrong here?

Hibernate needs an active `DataSource`/connection to validate that your `@Entity` mappings match the real database schema. If a mismatch exists (e.g., an entity field with no matching column) and `ddl-auto` is set to `validate`, the application fails to start immediately at this step with a clear schema error — much better than a confusing runtime failure discovered later.

Q5. Why are Security Filters loaded before the embedded server starts accepting real traffic?

To guarantee no request can ever be processed before authentication/authorization infrastructure is fully in place — the security filter chain must be completely assembled before Tomcat binds its port and starts accepting real connections.

Q6. What's the difference between @PostConstruct and CommandLineRunner in terms of WHEN they execute?

`@PostConstruct` runs per-bean, immediately after that specific bean's own dependencies are injected — potentially quite early in the overall startup sequence. `CommandLineRunner` runs only after the ENTIRE `ApplicationContext` has finished refreshing (every bean fully initialized), making it the very last hook before the application is considered ready.

Q7. Why might a Kubernetes readiness probe still fail even after the application's port is bound?

Port binding (Step 16) happens before all remaining beans finish initializing and before `@PostConstruct`/`CommandLineRunner` logic runs (Steps 17-19) — the readiness probe endpoint only returns healthy once `ApplicationReadyEvent`-level completeness is reached (Step 20-21), correctly preventing traffic from reaching a pod that's still finishing its startup work.

Q8. Describe the full lifecycle of a single HTTP request in a Spring Boot application.

Load balancer routes it to a pod's Tomcat → the servlet filter chain runs first (CORS, correlation ID, security authentication/authorization) → `DispatcherServlet` dispatches to the matching `@Controller` method via `HandlerMapping` → the request body is validated and deserialized → the Controller delegates to the Service layer within a `@Transactional` boundary → the Repository checks cache then the database → the transaction commits → a DTO is returned, serialized to JSON, and sent back through the filter chain to the client.

Q9. Where does a distributed cache like Redis fit into the request lifecycle?

Typically inside the Service or Repository layer, checked BEFORE hitting the database — on a cache hit, the database query is skipped entirely; on a miss, the database is queried and the result is stored in the cache for next time, exactly the pattern detailed in Section 27 onward.

Q10. Why does understanding the exact startup sequence matter for real production debugging?

It lets you pinpoint exactly WHERE a startup failure occurred (e.g., a schema validation error means Hibernate initialization/Step 12 failed, pointing at an entity-schema mismatch) rather than guessing, and it explains real production behaviors like why a pod briefly accepts connections before truly being ready, informing correct readiness-probe design.

23. What Is Caching and Why We Need It

23.1 What Caching Is

Caching stores a copy of expensive-to-compute or expensive-to-fetch data in a fast-access location, so repeated requests for the same data can be served instantly instead of redoing the expensive work every single time.

23.2 A Simple Real Example

Without Cache vs With Cache

Without cache: EVERY request for interest rates hits the database — 50ms each, every time, 10,000 times/day

— VS —

With cache: FIRST request hits the database (50ms), stores result in cache; next 9,999 requests served from cache (0.5ms each)

23.3 Why Caching Matters — Real Reasons

Reason	Real Impact
Reduces database load	Fewer repeated queries means the database can handle far more unique/necessary work
Improves response time	In-memory lookups (microseconds) vs disk-based database queries (milliseconds)
Reduces cost	Less database CPU/IO usage often means smaller, cheaper database infrastructure
Improves resilience	A brief database hiccup doesn't necessarily affect cached reads still being served

23.4 What Should — and Shouldn't — Be Cached

Good Candidates	Bad Candidates
Reference/lookup data (interest rates, product catalogs) — changes rarely	Real-time account balances — must always be current
Expensive computed results (credit score calculations)	Highly sensitive data without careful security/TTL controls
Data read far more often than it's written	Data requiring strict read-your-own-write consistency

Interview one-liner: "Caching trades a small amount of potential staleness for a large amount of speed and reduced load — the art of good caching is choosing exactly which data can tolerate that trade-off, and for how long."

24. Before Redis — What We Used Instead

24.1 Caching Approaches Before Redis Became Standard

Approach	How It Worked	Real Limitation
Local in-memory cache (HashMap, ConcurrentHashMap)	Store data directly in the application's own JVM heap memory	Per-instance only — in a multi-server deployment, each server has its own separate, inconsistent cache
EhCache	A mature, feature-rich local Java caching library with eviction policies	Still fundamentally per-JVM by default, same multi-instance consistency problem
Memcached	A distributed, simple key-value memory cache — an early real distributed caching solution	Only supports simple strings/objects, no rich data structures, no persistence, no built-in replication
Database query caching	Some databases offered built-in query result caching	Tightly coupled to the specific database, limited flexibility, often deprecated in modern DB versions

24.2 The Real Problem That Led to Redis's Popularity

As applications moved to running multiple instances behind a load balancer (Section 8.1) — especially with microservices and Kubernetes — a purely local, per-JVM cache became fundamentally broken: instance A's cache update was invisible to instances B, C, and D. Organizations needed a genuinely **shared, distributed** cache that every instance could read and write consistently.

24.3 Why Redis Specifically Won Out

- Extremely fast — an in-memory data store with sub-millisecond typical latency.
- Rich data structures beyond simple key-value (Section 26) — strings, hashes, lists, sets, sorted sets.
- Built-in persistence options (can survive a restart, unlike a pure in-memory-only cache).
- Native replication and clustering support for high availability at scale (Section 32).
- A simple, well-documented protocol with mature client libraries in every major language, including excellent native Spring Boot integration.

Interview tip: Redis didn't invent distributed caching — Memcached came first — but Redis's richer data structures, built-in persistence, and native replication/clustering made it the dominant modern choice for most real organizations' caching (and beyond-caching) needs.

25. Introduction to Redis

25.1 What Redis Is

Redis (REmote DIctionary Server) is an open-source, in-memory data store — used as a cache, a message broker (Pub/Sub), a session store, and even a lightweight primary database for certain use cases, thanks to its speed and versatile data structures.

25.2 Redis vs a Traditional Database — Core Difference

Aspect	Traditional RDBMS	Redis
Primary storage location	Disk (with memory caching internally)	Memory (RAM) primarily, with optional disk persistence
Data model	Structured tables, rows, columns, relationships	Simple key-value with rich value types (Section 26)
Query language	SQL — powerful, flexible querying/joining	Simple commands per data structure — no joins, no complex query language
Speed	Milliseconds (disk-backed)	Sub-millisecond (memory-backed)
Best fit	Complex relational data, strong consistency, long-term storage	Fast lookups, caching, session storage, real-time counters

Redis and a relational database are not competitors — real applications use BOTH together: PostgreSQL/MySQL as the durable system of record, Redis as the fast layer in front of it for frequently-accessed data.

26. Redis Data Structures

26.1 String — The Simplest Type

```
SET account:1001:balance "4500.00"  
GET account:1001:balance  
INCR page:views           # atomic increment – a simple, thread-safe counter  
SETEX session:abc123 1800 "user-data-json" # set with a 1800-second (30 min) expiry (TTL)
```

26.2 Hash — Storing Structured Objects

```
HSET account:1001 balance 4500.00 status "ACTIVE" customerId 55  
HGET account:1001 balance  
HGETALL account:1001      # retrieves all fields – often used to cache an entire object's fields together
```

26.3 List — Ordered Collection

```
LPUSH recent_transactions:1001 "txn-9921" # add to the front  
LRANGE recent_transactions:1001 0 9        # get the 10 most recent items
```

26.4 Set — Unique, Unordered Collection

```
SADD blocked_ips "203.0.113.44"  
SISMEMBER blocked_ips "203.0.113.44" # fast O(1) membership check – great for blocklists/allowlists
```

26.5 Sorted Set (ZSET) — Ranked Data

```
ZADD leaderboard 1500 "customer:55"  
ZADD leaderboard 2100 "customer:88"  
ZREVRANGE leaderboard 0 2 WITHSCORES # top 3 ranked entries – great for leaderboards, rate-limit windows
```

26.6 Real Data Structure Selection Table

Data Structure	Real Use Case
String	Simple cached values, atomic counters, session tokens
Hash	Caching an entire object (like an Account) as one grouped structure
List	Recent activity feeds, simple queues
Set	Blocklists, tags, uniqueness checks
Sorted Set	Leaderboards, rate limiting windows, priority queues

27. Redis Architecture — How It Actually Works Internally

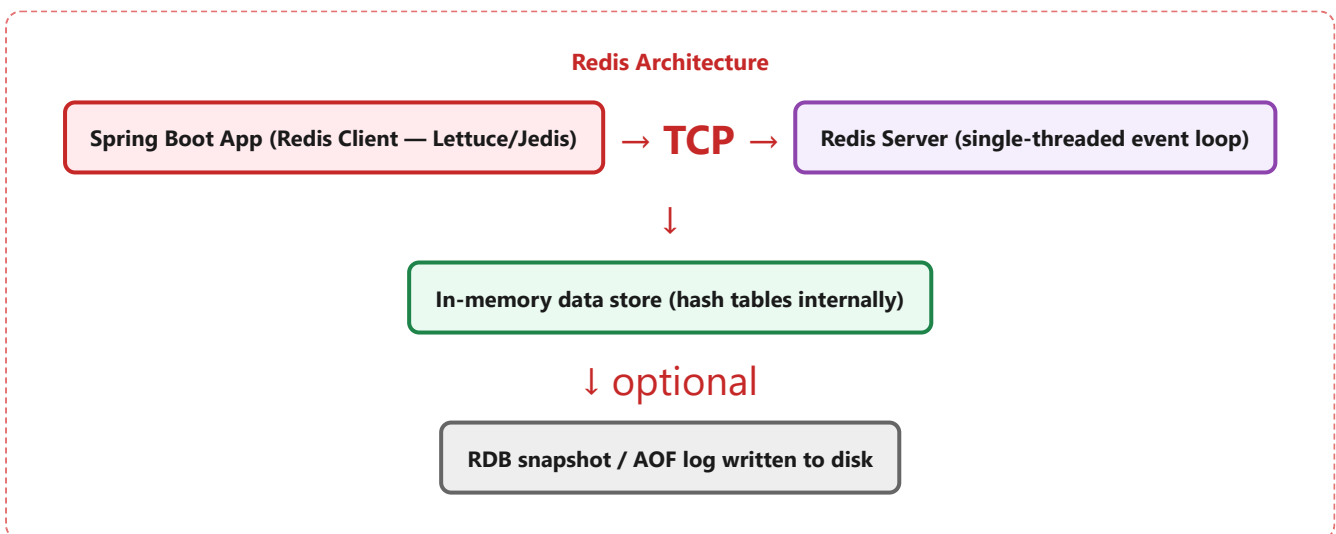
27.1 Single-Threaded Command Execution

Redis processes commands using a single main thread (per core, historically) — this sounds like a limitation, but because everything happens in memory (no disk I/O wait), it's extremely fast AND eliminates entire categories of concurrency bugs (no need for complex locking between commands).

27.2 In-Memory Storage With Optional Persistence

Persistence Option	How It Works
RDB (snapshotting)	Periodically saves a full point-in-time snapshot of the dataset to disk
AOF (Append Only File)	Logs every write operation to a file, replayed on restart — more durable, slightly slower
No persistence	Pure in-memory only — data is lost entirely on restart (acceptable for a pure, rebuildable cache)

27.3 Redis Client-Server Flow



28. Implementing Redis in Spring Boot — Step by Step

28.1 Step 1 — Add the Dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

28.2 Step 2 — Configure Connection Details

```
spring:
  data:
    redis:
      host: redis-cluster.prod.internal
      port: 6379
      password: ${REDIS_PASSWORD}
      timeout: 2000ms
  cache:
    type: redis
    redis:
      time-to-live: 3600000
```

28.3 Step 3 — Enable Caching and Configure the CacheManager

```
@SpringBootApplication
@EnableCaching
public class BankApplication { ... }

@Configuration
public class RedisConfig {
    @Bean
    public RedisCacheConfiguration cacheConfiguration() {
        return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(30))
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()));
    }
}
```

28.4 Step 4 — Use Declarative Caching Annotations

```
@Service
@CacheConfig(cacheNames = "accounts")
@RequiredArgsConstructor
public class AccountService {
    private final AccountRepository accountRepository;

    @Cacheable(key = "#id")
    public AccountDto findById(Long id) {
        return mapper.toDto(accountRepository.findById(id).orElseThrow());
    }

    @CacheEvict(key = "#id")
    public void closeAccount(Long id) {
        accountRepository.deleteById(id);
    }
}
```

This exact pattern — the full `@Cacheable` / `@CachePut` / `@CacheEvict` annotation set — is already covered in complete depth in both the Spring & Spring Boot guide (Section 52-53) and the JDBC/Hibernate/JPA guide; this section focuses specifically on the network/architecture context of how Redis fits into the request lifecycle (Section 21.1, Step 11).

28.5 Step 5 — Using RedisTemplate Directly (Beyond Simple Caching)

```
@Service
@RequiredArgsConstructor
public class RateLimiterService {
    private final RedisTemplate<String, String> redisTemplate;

    public boolean isAllowed(String apiKey, int maxRequestsPerMinute) {
        String key = "ratelimit:" + apiKey;
        Long count = redisTemplate.opsForValue().increment(key);
        if (count == 1) {
            redisTemplate.expire(key, Duration.ofMinutes(1)); // TTL set only on the FIRST request in this
window
        }
        return count <= maxRequestsPerMinute;
    }
}
```

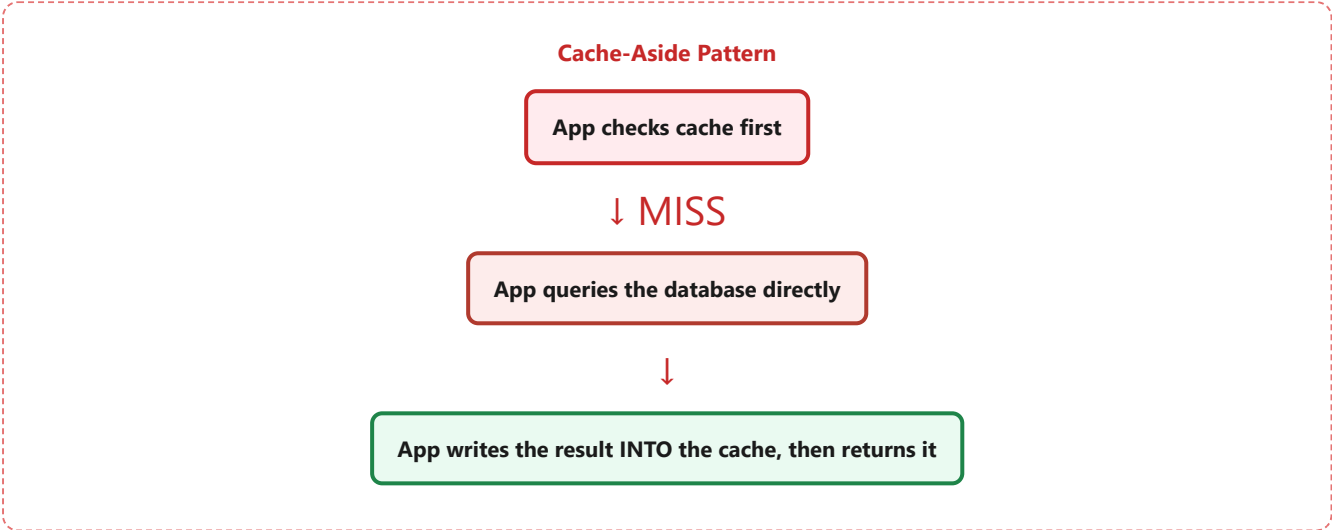
29. Redis Use Cases Beyond Simple Caching

Use Case	Redis Feature Used	Real Example
Session Storage	Hash/String with TTL	Spring Session backed by Redis, shared across all pods (Section 6.4)
Rate Limiting	Atomic INCR + TTL	API Gateway limiting requests per API key per minute
Distributed Locking	SETNX (set-if-not-exists) with TTL	Preventing two pods from processing the same batch record simultaneously
Leaderboards/Rankings	Sorted Set (ZSET)	Real-time ranking of top-performing sales branches
Pub/Sub Messaging	PUBLISH/SUBSCRIBE commands	Lightweight real-time notifications across service instances (not a durable replacement for Kafka)
Real-Time Counters	Atomic INCR/DECR	Live "active users" or "requests in flight" dashboards

Real scenario: "Our API Gateway uses Redis-backed atomic counters with a 1-minute TTL to enforce per-partner rate limits — a partner's 61st request within any rolling minute is rejected with a 429, using nothing more than an INCR command and an EXPIRE, no separate rate-limiting infrastructure needed."

30. Cache Eviction Policies & Caching Strategies (Cache-Aside, Write-Through, Write-Behind)

30.1 Cache-Aside (Lazy Loading) — The Most Common Pattern



This is exactly what Spring's `@Cacheable` implements automatically — the application code (or Spring's proxy, in this case) is responsible for populating the cache on a miss. Simple, and the most widely used real-world pattern.

30.2 Write-Through

Every WRITE goes to the cache AND the database together, synchronously, as one operation — reads are always guaranteed fresh from the cache, at the cost of slightly slower writes (since both systems must be updated before the write is considered complete).

30.3 Write-Behind (Write-Back)

Writes go to the cache immediately (fast), and the database update happens asynchronously afterward — faster writes, but with a real risk of data loss if the cache fails before the delayed database write completes. Used sparingly, only where that risk is genuinely acceptable.

30.4 Real Comparison Table

Strategy	Read Speed	Write Speed	Data Safety	Real Use
Cache-Aside	Fast after first miss	Normal (writes go straight to DB)	Safe — DB is always source of truth	The default choice for most real caching (<code>@Cacheable</code>)
Write-Through	Always fast	Slower (writes both cache+DB)	Safe — both stay in sync immediately	Data needing guaranteed freshness on every read

Write-Behind	Always fast	Very fast	Riskier — possible data loss window	High-write-volume, loss-tolerant scenarios (e.g., analytics counters)
--------------	-------------	-----------	-------------------------------------	---

30.5 Eviction Policies — What Happens When Redis Memory Fills Up

Policy	Behavior
noeviction	Rejects new writes once full — safest for non-cache uses, risky for a busy cache
allkeys-lru	Evicts the Least Recently Used key across ALL keys — the standard default for a pure cache
volatile-lru	Evicts LRU only among keys that have a TTL set, protecting permanent keys
allkeys-random	Evicts a random key — cheap to compute, rarely the best choice

31. Distributed Caching Challenges — Consistency & Invalidation

31.1 "There Are Only Two Hard Things in Computer Science..."

The classic saying finishes: "...cache invalidation and naming things." Cache invalidation is genuinely one of the hardest real-world problems — knowing exactly WHEN cached data has become stale and needs refreshing is deceptively difficult to get completely right.

31.2 The Real Bypass Bug — A Common Production Incident

If data is updated through a path that DOESN'T go through the application's normal service layer (a direct SQL fix, a manual database correction, a different microservice writing to the same table), Spring's `@CacheEvict` / `@CachePut` never fires — Redis keeps serving the OLD, now-incorrect cached value until its TTL naturally expires.

Real scenario: "A support engineer manually corrected a customer's card limit directly via a database script during an incident — bypassing the application layer entirely meant Redis never got evicted, and the customer kept seeing the old limit in the app for another 30 minutes until the TTL expired. We fixed the PROCESS by requiring all manual data corrections to go through an authenticated internal API endpoint, which properly triggers cache eviction, never direct SQL against production."

31.3 Consistency Trade-Offs — TTL vs Explicit Invalidation

Approach	How	Trade-Off
TTL-only (time-based expiry)	Cache entries automatically expire after a fixed duration	Simple, but data can be stale for up to the full TTL duration after any change
Explicit invalidation (<code>@CacheEvict</code>)	Application code explicitly removes/updates the cache entry the moment data changes	Immediately consistent, but ONLY if every single write path remembers to do this correctly
Both together (real best practice)	Explicit invalidation for known write paths, PLUS a safety-net TTL for anything missed	Best of both — immediate consistency in the common case, bounded staleness worst case

31.4 Cache Stampede — Another Real Distributed Caching Problem

If a very popular cache key expires and MANY concurrent requests all experience a cache miss simultaneously, they can all hammer the database at the exact same moment trying to recompute/refetch the same data — a "thundering herd" that can itself cause an outage.

Real fixes include: using a distributed lock so only ONE request recomputes the value while others wait briefly, or proactively refreshing popular keys slightly BEFORE they actually expire ("early refresh") rather than waiting for a hard expiry followed by a stampede of misses.

32. Redis High Availability — Replication, Sentinel & Cluster

32.1 Master-Replica Replication

A Redis Master handles all writes, and one or more Replicas continuously receive a copy of the data — replicas can serve read traffic (spreading load) and provide a failover target if the master goes down.

Master-Replica Setup

Redis Master (handles writes)

↓ replicates data to

Replica 1

Replica 2

32.2 Redis Sentinel — Automatic Failover

Sentinel processes monitor the Master and Replicas, and automatically PROMOTE a replica to become the new Master if the original Master fails — providing automated high availability without manual intervention.

32.3 Redis Cluster — Horizontal Scaling

Redis Cluster shards data across multiple master nodes (each responsible for a subset of the total "hash slots"), allowing the total dataset size and write throughput to scale far beyond what a single Redis instance can handle — each shard can also have its own replicas for HA.

Setup	Solves	Real Use
Standalone Redis	Simple caching, single point of failure acceptable	Small applications, non-critical caching
Master-Replica + Sentinel	High availability, automatic failover	Production caching where downtime must be minimized
Redis Cluster	Horizontal scale for very large datasets/throughput	Large-scale, high-traffic production systems

Real scenario: "Our production Redis runs as a managed Azure Cache for Redis instance with built-in replication and automatic failover — when the primary node had a brief hardware issue, failover to a replica happened in under 30 seconds, with our application's Lettuce client automatically reconnecting to the new primary, and only a brief handful of cache misses (not an outage) resulted."

33. Redis Security Best Practices

Practice	Why
Require authentication (requirepass / ACLs)	Prevents unauthorized access if the network perimeter is ever breached
Never expose Redis directly to the public internet	Redis was not originally designed with internet-facing security as a primary concern — always keep it inside a private network/VPC
Use TLS for connections	Encrypts data in transit between the application and Redis, especially important for sensitive cached data
Use Redis ACLs for fine-grained permissions	Restrict which commands/keys a given application/service can access (e.g., a reporting service gets read-only access)
Never cache raw sensitive data unencrypted	Card numbers, passwords, or full PII should never be cached in plain text, even in a supposedly "internal" cache
Set appropriate memory limits and eviction policies	Prevents Redis from being exhausted/crashed by unbounded key growth

Real-world reminder: Redis historically had NO authentication or encryption by default — several real, widely publicized security incidents occurred from Redis instances left accidentally exposed to the public internet with no password. Always treat Redis as sensitive internal infrastructure, never a publicly-reachable service.

34. Redis Real-Time Issues & Scenarios

Issue	Cause	Real Solution
Stale data shown to users	Cache eviction bypassed by a direct/manual data change	Route all writes through the application layer; add a safety-net TTL (Section 31.3)
Sudden database load spike when a popular key expires	Cache stampede — many concurrent misses at once	Distributed lock around recomputation, or proactive early refresh (Section 31.4)
Users randomly logged out under load	Sessions stored in-memory per-instance instead of shared Redis	Use Spring Session backed by Redis (Section 6.4)
Redis memory usage growing unbounded	No TTL set on cache entries, or no eviction policy configured	Always set a sensible TTL on cache entries; configure allkeys-lru or similar eviction policy
Application errors after a Redis failover	Client not configured to automatically reconnect/retry	Use a modern client (Lettuce, Spring Boot's default) with automatic reconnection and sensible timeouts
Rate limiter allows more requests than intended under high concurrency	Non-atomic check-then-increment logic (race condition)	Use Redis's atomic INCR command directly, never a separate GET-then-SET pair
Sensitive data exposed via an exported Redis dump	PII/card data cached in plain text	Never cache raw sensitive fields; mask/encrypt or simply exclude them from any cached payload
Cross-instance cache inconsistency intermittently reported	A mix of local (per-JVM) caching AND Redis caching used inconsistently across the codebase	Standardize on Redis (via @Cacheable) as the ONLY shared cache mechanism for any data needing cross-instance consistency

35. Redis Interview Questions & Answers

Q1. What is caching, and why is it important in real applications?

Caching stores a copy of expensive-to-fetch/compute data in a fast-access location, reducing database load, improving response times, and reducing infrastructure cost — the trade-off is potential staleness, which must be managed deliberately per use case.

Q2. What did teams use for caching before Redis, and what problem forced a change?

Local in-memory caches (HashMap, EhCache) and Memcached. The real problem was multi-instance deployments — a purely local cache is per-JVM and becomes inconsistent across instances behind a load balancer, forcing the industry toward genuinely shared, distributed caching solutions like Redis.

Q3. Why is Redis so fast?

It stores data primarily in memory (RAM) rather than disk, and processes commands via a highly optimized single-threaded event loop that avoids locking overhead between commands — both together achieve typical sub-millisecond latency.

Q4. Name Redis's core data structures and a real use case for each.

String (simple values, counters), Hash (caching a whole object's fields together), List (recent activity feeds), Set (blocklists/uniqueness checks), Sorted Set/ZSET (leaderboards, rate-limiting windows) — choosing the right structure for the access pattern is a key Redis design skill.

Q5. Explain the Cache-Aside pattern and how Spring's @Cacheable implements it.

Cache-Aside checks the cache first; on a miss, it queries the source of truth (the database) and populates the cache before returning the result. @Cacheable implements exactly this — on a cache hit it skips the method body entirely; on a miss it executes the method and stores the result.

Q6. What is cache invalidation, and why is it considered one of the hardest problems in computing?

Cache invalidation is knowing exactly when cached data has become stale and must be refreshed/removed. It's hard because any write path that bypasses the normal cache-eviction logic (like a direct database fix) leaves stale data being served, often silently, until a TTL eventually expires.

Real scenario: a manual database correction during an incident bypassed @CacheEvict, leaving a customer seeing an outdated card limit until the TTL expired — fixed by requiring all manual fixes to go through the application layer.

Q7. What is a cache stampede, and how do you prevent it?

When a popular cache key expires, many concurrent requests can all experience a cache miss simultaneously and hammer the database at once trying to recompute the same value. Prevented via a distributed lock (only one request recomputes while others wait) or by proactively refreshing hot keys before they actually expire.

Q8. How does Redis provide high availability?

Master-Replica replication keeps copies of data on additional nodes; Redis Sentinel monitors the setup and automatically promotes a replica to master on failure; Redis Cluster additionally shards data across multiple masters for horizontal scaling of both capacity and throughput, each shard optionally having its own replicas.

Q9. Why should Redis never be exposed directly to the public internet?

Redis was not originally designed with internet-facing security as a primary concern (historically no default authentication/encryption) — several real security incidents occurred from publicly exposed, unauthenticated Redis instances. It should always run inside a private network, with authentication and TLS enabled regardless.

Q10. Describe a real production Redis-related issue you've resolved.

A rate limiter occasionally allowed more requests than intended under high concurrency due to a non-atomic "read count, check, then increment" pattern racing across concurrent requests. Fixed by using Redis's atomic INCR command directly (which is inherently safe under concurrency, since Redis processes commands one at a time) instead of a separate GET-then-SET sequence.

36. Final Summary & Cheat Sheet

Web & Network Basics: the client-server model, DNS resolving names to IPs, ports identifying specific applications on a server, HTTPS encrypting traffic, and cookies/sessions solving HTTP's fundamental statelessness — with modern APIs increasingly favoring stateless JWTs over server-side sessions for easier multi-instance scaling.

Spring Boot Startup: a strict 21-step sequence — JVM boot → Spring container creation → configuration loading → component scanning and dependency injection → auto-configuration wiring infrastructure (DataSource, JPA, Security) → embedded server starting and binding its port → remaining bean lifecycle hooks (@PostConstruct, CommandLineRunner) → ApplicationReadyEvent, only THEN is the app truly ready for traffic.

The Request Lifecycle: every request flows through a load balancer → Tomcat → the servlet filter chain (security first) → DispatcherServlet → Controller → Service (@Transactional) → Repository (checking Redis before the database) → back out through serialization and the filter chain to the client.

Redis & Caching: caching trades a bounded amount of staleness for major speed/cost benefits; Redis became the dominant distributed cache because a purely local, per-JVM cache breaks down the moment an application runs on more than one instance; Cache-Aside (via @Cacheable) is the standard real-world pattern, with careful attention to invalidation correctness, cache stampede prevention, and high-availability replication for production-grade reliability.

36.1 Quick Cheat Sheet

Term	One-Line Meaning
DNS	Translates domain names into IP addresses
Cookie	Small client-side data sent automatically with every request to a domain
Session	Server-side user state, referenced by a client-held session ID
JWT	Self-contained, signed, stateless token replacing server-side sessions
ApplicationContext	Spring's IoC container, created early in the startup sequence
Auto Configuration	Classpath-driven, conditional bean setup eliminating manual wiring
ApplicationReadyEvent	Signals the application is FULLY started and ready for traffic
Cache-Aside	Check cache first, populate it on a miss — the standard @Cacheable pattern
Cache Stampede	Many concurrent cache misses hammering the database at once
Redis Sentinel	Monitors Redis and automates failover to a replica
Redis Cluster	Shards data across multiple Redis masters for horizontal scale

Use this guide alongside the Spring & Spring Boot guide, the API Development guide, the Microservices guide, the JDBC/Hibernate/Spring Data JPA guide, and the Observability & Monitoring guide — together they cover the complete real-world Java full-stack developer journey from the network fundamentals underneath every request, through exactly how a Spring Boot application boots and serves that request, to the caching layer that keeps it all fast at scale.